

Lab 5: Synchronous vs Asynchronous execution

Phillip Pinter

March 28, 2024

1 Introduction

The goal of this lab is to compare the difference in performance between executing the program synchronously or asynchronously.
Synchronously means that the program executes step-by-step, so when the CPU calls a kernel function, the CPU waits until the GPU is done executing the kernel and only after that, the CPU continues with the program.
Asynchronously means that the CPU can continue with the program while the GPU is working. This enables the CPU to call multiple kernels (or other asynchronous functions) that can run in parallel. It is important to make sure that you still synchronize at crucial points to avoid situations where, for example, a kernel needs the result of another kernel that hasn't finished executing yet.

2 Results: Synchronous vs Asynchronous

To test the performance difference between synchronous vs asynchronous, a program is made which is based on the program of lab 2, finding the maximum value in an array. The functionality of that program is expanded to also be able to find the minimum value, sum and product of all values in the array. This can be done by changing the initial value of the result variable because for finding the maximum value you start off with the smallest possible number and vice versa for the minimum value. For the sum you start with zero and for the multiplication you start with one. After that you change the logic of what happens with the two elements a thread is comparing when doing reduction. For finding the minimum or maximum value, you put the smallest or biggest of the two elements in the first element. For the sum and multiplication you put the result of adding or multiplying the two values in the first element. This will result in four kernels, each doing one of the operations.

After this, 2 functions are made which are both timed from the beginning to the end of the function: one for the synchronous execution and one for the asynchronous. Both functions have the same structure:
Initialize the array for the first operation (finding the maximum) → copy the data into the device memory → run the kernel → copy the result back to the host variable. This is done for all four operations in sequence, so the arrays aren't initialized after each other, the array of the second array is initialized after the kernel call of the first one.
Since kernel calls are by default asynchronous, the device is synchronized between each operation in the synchronous implementation to make sure that the next array only starts initializing after the current kernel is done executing. The "cudaMemcpy()" function is already synchronous so this will stay the same.
For the asynchronous implementation the device isn't synchronized between each operation. To copy the memory from the host to the device and vice versa, the function "cudaMemcpyAsync()" is used instead. This will assure that the CPU isn't blocked from proceeding with the next instruction while the memory is copied. At the end of this implementation the device is synchronized to make sure all kernels are done executing before stopping the timer.

The arrays contain random values between 1 and 10 but the same seed was used for the random number generation to get the same arrays for both implementations even though they are all initialized separately (not copied) with "random" numbers. Running the program gave following results (which were all verified to be correct):

```
ResultMax:  10
ResultMin:  1
ResultSum:  1374
ResultMult: 5832000
Synchronous execution time: 198.233 milliseconds.
ResultMax:  10
ResultMin:  1
ResultSum:  1374
ResultMult: 5832000
Asynchronous execution time: 0.9499 milliseconds.
```



This shows an incredible performance increasement for using asynchronous programming, in this case more than 200 times as fast. This is mainly due to the fact that the CPU can continue with its instructions while the GPU is working and the usage of the GPU's resources is much more optimized (multiple kernels are being executed instead of only one at a time).